

УДК 519.688

Сравнение алгоритмов программного соперника в игровом приложении «Коридор»

А. О. Карпунин

E-mail: aleksandrkarpunin0889@gmail.com

Ярославский государственный университет им. П.Г. Демидова

Аннотация

В статье рассматривается разработка модуля стратегического поведения программного соперника для настольной игры «Коридор». Описана игровая модель, функция оценки позиции и три алгоритма выбора хода: метод минимакса, метод альфа-бета-отсечений и метод Монте-Карло. Для оценки качества реализованных подходов проведено автоматическое сравнение алгоритмов на поле 11×11 . В результате эксперимента метод альфа-бета-отсечений демонстрирует наибольшую долю побед, а наибольшее время хода необходимо методу Монте-Карло.

Ключевые слова: игра «Коридор», программный соперник, алгоритмы поиска, Монте-Карло, функция оценки

Введение

Логические настольные игры удобны для исследования алгоритмов искусственного интеллекта, поскольку состояние партии можно строго описать, а множество допустимых действий формируется по заранее заданным правилам. Одной из таких игр является «Коридор» — игра с полной информацией для двух игроков. Каждый игрок управляет фишкой и стремится первым дойти до противоположной стороны поля. В процессе игры игроки могут устанавливать перегородки, усложняющие путь соперника [1].

В «Коридоре» каждый игрок выбирает между продвижением к цели и изменением структуры игрового поля. Поэтому программный соперник должен учитывать не только собственное расстояние до финишной линии, но и возможность замедлить соперника с помощью стен. Полный перебор дерева игры быстро становится слишком трудозатратным, поэтому алгоритмы выбора хода требуют ограничений глубины, эвристик и оценки промежуточных позиций.

Цель работы — разработать модуль программного соперника для игрового приложения «Коридор» и сравнить несколько алгоритмов

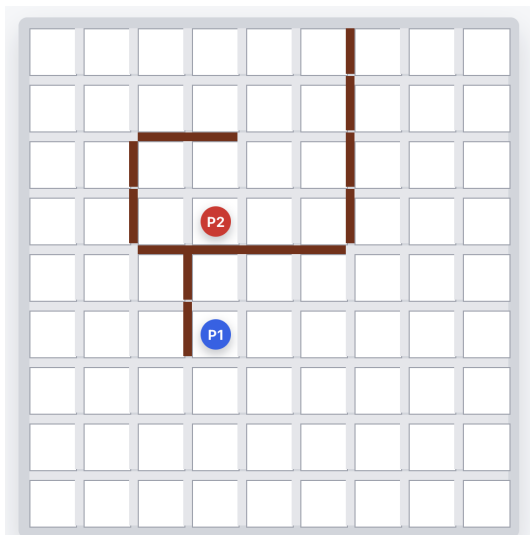


Рис. 1. Игровое поле «Коридора»

выбора хода: метод минимакса, метод альфа-бета-отсечений и метод Монте-Карло.

Правила и модель игры

В настольной игре «Коридор» используется квадратное поле 9×9 , однако в эксперименте будет использовано поле 11×11 . В начале партии фишки игроков располагаются на противоположных сторонах поля: первый игрок — в центре нижней строки, второй — в центре верхней. Цель каждого игрока — первым достичь любой клетки своей целевой линии.

За один ход игрок может выполнить одно из двух действий [1]:

- Передвинуть фишку. Фишка передвигается на смежную клетку, если переход не заблокирован стеной. Если соседняя клетка занята фишкой соперника и за ней нет ни перегородки, ни границы поля, фишка может перепрыгнуть через соперника.
- Установить стену. Стена занимает пространство между клетками, имеет длину две клетки и может быть расположена горизонтально или вертикально. Установка стены допускается, если у игрока ещё остались перегородки, новая стена не пересекается

с уже установленными и после её установки у каждого игрока остаётся хотя бы один маршрут к своей целевой линии.

Для проверки существования маршрута используется поиск в ширину от текущей позиции фишки до ближайшей клетки целевой строки.

В рассматриваемой модели множество допустимых ходов формируется из двух частей. Первая часть — перемещения фишки, получаемые проверкой направлений хода с текущей клетки и обработкой возможных прыжков через соперника. Вторая часть — варианты установки стен около кратчайшего пути соперника. Такой выбор используется потому, что стена полезна прежде всего тогда, когда меняет текущий лучший маршрут соперника; перегородка вдали от этого маршрута часто не увеличивает длину пути и только расширяет перебор.

Состояние игры можно представить набором данных:

- расположение фишек игроков;
- расположение стен;
- количество оставшихся стен у каждого игрока;
- номер игрока, который должен сделать следующий ход.

Такое представление позволяет использовать классические алгоритмы поиска в дереве игры [2]. Узел дерева соответствует состоянию партии, а ребро — допустимому ходу. Лист дерева оценивается численно с помощью функции оценки позиции.

Функция оценки позиции

Поскольку полный перебор партии невозможен, алгоритмам требуется приближённая оценка промежуточного состояния. В данной работе оценка позиции строится как линейная комбинация нескольких признаков:

$$F(s) = \sum_{k=1}^7 \lambda_k f_k(s),$$

где s — состояние игры, $f_k(s)$ — числовые признаки игровой позиции, а λ_k — соответствующие им веса. Чем больше итоговое значение $F(s)$, тем выгоднее позиция для программного соперника.

Основной признак связан с кратчайшими путями игроков до целевых строк. Если путь алгоритма до цели короче пути соперника, позиция считается более выгодной. Дополнительно учитываются мобильность фишек, оставшиеся стены, продвижение к целевой строке

и специальные эндшпильные признаки, которые усиливают оценку позиций, близких к победе или поражению. Используемые признаки приведены в таблице 1.

Таблица 1. Признаки функции оценки позиции

k	Признак $f_k(s)$	Смысл признака
1	$d_p - d_a$	Разность кратчайших путей соперника и алгоритма до целевых строк
2	mob_a	Количество допустимых перемещений фишкой алгоритма
3	$-mob_p$	Штраф за количество допустимых перемещений фишкой соперника
4	$w_a - w_p$	Преимущество алгоритма по числу оставшихся стен
5	$prog_a - prog_p$	Разность продвижения игроков к своим целевым строкам
6	$e(d_a)$	Бонус за близость алгоритма к победе
7	$-e(d_p)$	Штраф за близость соперника к победе

Здесь d_a и d_p — длины кратчайших путей алгоритма и соперника, w_a и w_p — количество оставшихся стен. Функция $e(d)$ задаёт эндшпильную поправку:

$$e(d) = \begin{cases} 10, & d \leq 1, \\ 4, & d = 2, \\ 1, & d = 3, \\ 0, & d > 3. \end{cases}$$

То есть позиция, в которой алгоритму остаётся один ход до победы, получает большой положительный вклад, а позиция, где близок к победе соперник, получает симметричный штраф.

Начальные веса были заданы вручную на основе смысла признаков:

$$\lambda_1 = 65, \quad \lambda_2 = 4, \quad \lambda_3 = 3, \quad \lambda_4 = 18, \quad \lambda_5 = 6, \quad \lambda_6 = 165, \quad \lambda_7 = 175.$$

Наибольшие веса имеют эндшпильные признаки и разность кратчайших путей, поскольку для игры «Коридор» особенно важно не

только продвигать собственную фишку, но и вовремя замечать угрозу быстрого выхода соперника к целевой строке.

При оценке стен дополнительно используется локальная эвристика стратегической ценности. После временного применения стены на копии поля вычисляется, насколько изменились кратчайшие пути игроков:

$$wallImpact = (d_p^{after} - d_p^{before}) \cdot 100 - \max(0, d_a^{after} - d_a^{before}) \cdot 45.$$

Таким образом, стена получает высокий приоритет, если она увеличивает путь соперника, и штрафует, если одновременно ухудшает путь самого алгоритма. Это используется при сортировке ходов и при поиске защитного хода в эндшпиле.

Также реализован механизм коррекции весов по результатам партий самообучения — автоматических партий, в которых программные соперники играют друг против друга без участия человека. Для каждого хода алгоритма сохраняется обучающий образец: значения признаков до хода и после него. После завершения партии вычисляется изменение каждого признака $\Delta f_k = f_k^{after} - f_k^{before}$, после чего формируется суммарный сигнал:

$$g_k = \sum_{\text{ходы алгоритма}} reward \cdot \Delta f_k, \quad reward = \begin{cases} +1, & \text{алгоритм победил,} \\ -1, & \text{алгоритм проиграл.} \end{cases}$$

Если $g_k > 0$, увеличение соответствующего признака чаще встречалось в победных партиях, поэтому вес признака увеличивается на один шаг. Если $g_k < 0$, вес уменьшается. Новое значение дополнительно ограничивается минимальной и максимальной границей, чтобы один признак не начал подавлять остальные:

$$\lambda_k^{new} = \begin{cases} \lambda_k^{\min}, & \lambda'_k < \lambda_k^{\min}, \\ \lambda_k^{\max}, & \lambda'_k > \lambda_k^{\max}, \\ \lambda'_k, & \lambda_k^{\min} \leq \lambda'_k \leq \lambda_k^{\max}. \end{cases}$$

После обучения веса сохраняются в файл конфигурации и используются в последующих экспериментах. На момент проведения экспериментов сохранённые веса имели вид:

$$\begin{aligned} \lambda_1 &= 120, & \lambda_2 &= 7, & \lambda_3 &= 12, & \lambda_4 &= 8, \\ \lambda_5 &= 25, & \lambda_6 &= 186, & \lambda_7 &= 170. \end{aligned}$$

По сравнению с начальными значениями сильнее всего выросли веса разности путей, ограничения мобильности соперника и продвижения к целевой строке. Это соответствует наблюдаемой стратегии игры: алгоритм чаще выбирает позиции, где он не просто приближается к победе сам, но и вынуждает соперника тратить дополнительные ходы на обход стен.

Метод минимакса

Метод минимакса действует по следующему принципу: алгоритм выбирает ход, максимизирующий оценку позиции, а для игрока — минимизирующий [2]. Поиск строится до заданной глубины, после чего листья дерева оцениваются с применением функции оценки $F(s)$.

Недостаток метода заключается в том, что число просматриваемых состояний растёт экспоненциально с глубиной поиска. Поэтому используется ограничение глубины и предварительная сортировка ходов: сначала рассматриваются перемещения, уменьшающие путь к цели, и стены, увеличивающие путь соперника.

В реализации алгоритм работает с итеративным углублением: сначала строится дерево небольшой глубины, затем глубина постепенно увеличивается, пока не достигнут заданный предел или не истёк лимит времени. Такой подход позволяет вернуть лучший найденный ход даже в том случае, если полный просмотр очередного уровня не был завершён. Для повторяющихся состояний используется кэш оценок, ключом которого является состояние доски, глубина и тип узла. Это снижает число повторных вычислений в ситуациях, когда одна и та же позиция достигается разными последовательностями ходов.

Перед рекурсивным просмотром множество допустимых ходов сортируется. В начало списка попадают немедленные победные ходы, затем перемещения, улучшающие кратчайший путь алгоритма, и стены с высоким значением *wallImpact*. После этого число рассматриваемых кандидатов ограничивается в соответствии с уровнем сложности. Поэтому метод минимакса является не полным перебором всех ходов до заданной глубины, а эвристически направленным поиском по наиболее перспективной части дерева. Общая схема выбора оценки приведена в алгоритме 1.

Алгоритм 1: Метод минимакса с итеративным углублением

```

1  Функция minimax(s, d, isMax):
2      if в кеше есть значение для (s, d, isMax) then
3          |   return значение из кеша;
4      end
5      if terminal(s) then
6          |   return терминальную оценку позиции;
7      end
8      if d = 0 then
9          |   return evaluate(s);
10     end
11     p ← игрок, которому принадлежит ход в узле s;
12     M ← getMoves(s, p);
13     if isMax then
14         |   best ←  $-\infty$ ;
15         foreach m ∈ M do
16             |   best ←
17                 |   max(best, minimax(apply(s, m), d - 1, false));
18         end
19     else
20         |   best ←  $+\infty$ ;
21         foreach m ∈ M do
22             |   best ← min(best, minimax(apply(s, m), d - 1, true));
23         end
24     end
25     сохранить best в кеше для (s, d, isMax);
26     return best;
27 return m*

```

Метод альфа-бета-отсечений

Метод альфа-бета-отсечений — оптимизированная версия метода минимакса. Он вычисляет тот же результат при том же порядке листьев, но отбрасывает ветви, которые не могут повлиять на итоговый выбор [2]. Для этого используются две границы: α — лучшая уже найденная оценка для максимизирующего игрока, и β — лучшая оценка для минимизирующего игрока. Схема поиска с отсечениями показана в алгоритме 2.

Алгоритм 2: Метод альфа-бета-отсечений

```

1  Функция alphabeta( $s, d, \alpha, \beta, isMax$ ):
2      if terminal( $s$ ) then
3          |   return терминальную оценку позиции;
4      end
5      if  $d = 0$  then
6          |   if isNoisyPosition( $s$ ) then
7              |   return quiescence( $s, \alpha, \beta, isMax, 2$ );
8              else
9                  |   return  $F(s)$ ;
10             end
11      end
12       $M \leftarrow \text{getMoves}(s)$ ;
13      if isMax then
14          |    $best \leftarrow -\infty$ ;
15          |   foreach  $m \in M$  do
16              |    $v \leftarrow \text{alphabeta}(\text{apply}(s, m), d - 1, \alpha, \beta, \text{false})$ ;
17              |    $best \leftarrow \max(best, v)$ ;
18              |    $\alpha \leftarrow \max(\alpha, best)$ ;
19              |   if  $\beta \leq \alpha$  then
20                  |   обновить ходы-отсекатели и таблицу удачных
21                      |   ходов;
22                      |   break;           // отсечение в MAX-узле
23              |   end
24          |   end
25      else
26          |    $best \leftarrow +\infty$ ;
27          |   foreach  $m \in M$  do
28              |    $v \leftarrow \text{alphabeta}(\text{apply}(s, m), d - 1, \alpha, \beta, \text{true})$ ;
29              |    $best \leftarrow \min(best, v)$ ;
30              |    $\beta \leftarrow \min(\beta, best)$ ;
31              |   if  $\beta \leq \alpha$  then
32                  |   обновить ходы-отсекатели и таблицу удачных
33                      |   ходов;
34                      |   break;           // отсечение в MIN-узле
35              |   end
36          |   end
37      end
38      return  $best$ ;

```

То, насколько метод альфа-бета-отсечений будет эффективно отсекаать ветки, сильно зависит от порядка их рассмотрения. Чем раньше будут рассмотрены более перспективные ходы, тем чаще будут происходить отсечения. Поэтому в реализации используются эвристики сортировки и запоминание ходов, которые ранее приводили к отсечениям. Благодаря этому алгоритм успевает рассмотреть дерево на большую глубину, чем метод минимакса за то же выделенное время.

Для ускорения поиска метод альфа-бета-отсечений использует несколько дополнительных эвристик. Первая из них — отсекающие ходы: если некоторый ход на данной глубине уже приводил к отсечению $\beta \leq \alpha$, он считается потенциально сильным и в похожих узлах рассматривается раньше остальных. Вторая эвристика хранит накопленную статистику успешных отсечений в таблице удачных ходов. Чем чаще ход вызывал отсечение, тем больший приоритет он получает при последующей сортировке.

Отдельная доработка связана с позициями около конца партии. Если на предельной глубине один из игроков находится не дальше двух ходов от цели, позиция считается острой. В таком случае алгоритм не сразу применяет статическую оценку $F(s)$, а дополнительно рассматривает немедленные победные ходы и наиболее ценные защитные стены. Это уменьшает вероятность ситуации, когда поиск остановился прямо перед решающим ходом и неверно оценил позицию как безопасную.

Метод Монте-Карло

Метод Монте-Карло выбирает ход на основе многократных симуляций продолжения партии. Он не стремится равномерно раскрыть дерево до фиксированной глубины, а оценивает ходы статистически [3]. Общая схема статистического выбора хода приведена в алгоритме 3.

Для каждого доступного хода выполняются симуляции, в которых дальнейшие действия выбираются с учётом функции оценки позиции. Чем чаще ход приводит к выигрышным или позиционно выгодным продолжениям, тем выше его итоговый рейтинг.

Поиск методом Монте-Карло организован как дерево посещённых состояний. Каждый узел хранит число посещений n_v и число успешных симуляций w_v . При выборе следующего узла используется

Алгоритм 3: Метод Монте-Карло**Input:** состояние s , временной лимит T , лимит итераций N **Output:** выбранный ход

```

1  $r \leftarrow$  корневой узел с состоянием  $s$ ;
2 while не истёк лимит  $T$  и итераций  $< N$  do
3    $v \leftarrow \text{select}(r)$ ; // спуск по UCT
4   if  $\neg \text{terminal}(v.s)$  then
5      $v \leftarrow \text{expand}(v)$ ; // новый дочерний узел
6   end
7    $\text{result} \leftarrow \text{simulate}(v, \text{rolloutDepth})$ ;
8    $\text{backpropagate}(v, \text{result})$ ;
9   if  $r.n > 300$  и  $\max_c(w_c/n_c) > 0.97$  then
10    break;
11  end
12 end
13 return  $\arg \max_{c \in \text{children}(r)} (n_c + w_c/n_c)$ 

```

показатель UCT:

$$\text{UCT}(v) = \frac{w_v}{n_v} + C \sqrt{\frac{\ln n_{\text{parent}}}{n_v}} + h(v).$$

Первое слагаемое отражает накопленную долю успешных симуляций, второе отвечает за исследование редко посещённых узлов, а $h(v)$ добавляет небольшую эвристическую поправку на основе функции оценки позиции. В реализации эта поправка вычисляется как $0,15 \cdot \tanh(F(s_v)/500)$, поэтому она помогает направлять поиск, но не подавляет статистику симуляций.

Одна итерация метода Монте-Карло состоит из четырёх этапов: выбора узла по UCT, расширения дерева новым дочерним состоянием, симуляции партии и обратного распространения результата. Во время симуляции алгоритм не делает полностью случайные ходы: сначала проверяется немедленная победа, затем возможность поставить защитную стену, если соперник близок к цели, а в остальных случаях ход выбирается из ограниченной выборки по эвристической оценке. После завершения симуляции статистика посещений и результатов обновляется на всём пути от выбранного узла к корню.

Таблица 2. Доля побед алгоритмов на поле 11×11

Алгоритм	Партий	Победы	Лимит	Доля побед
Альфа-бета-отсечения	150	121	2	80,67 %
Монте-Карло	150	88	3	58,67 %
Минимакс	150	84	4	56,00 %
Случайный	150	0	5	0,00 %

Метод Монте-Карло может работать без точного перебора дерева до заданной глубины, но у него есть существенный недостаток: для устойчивого выбора требуется провести много симуляций, что подразумевает значительные временные затраты.

Методика сравнения

Эксперимент проводился на поле 11×11 . Для каждой пары алгоритмов запускалось 50 автоматических партий с чередованием роли первого игрока и максимальной сложностью. Во время сравнения обучение весов функции оценки не выполнялось, чтобы параметры оставались фиксированными. В эксперименте также участвовал алгоритм случайного выбора хода.

Собирались две группы метрик. Первая группа описывает качество игры: количество побед, поражений и партий, завершённых по лимиту ходов. Вторая группа описывает вычислительную стоимость: среднее время выбора хода, глубину поиска и число просмотренных состояний.

Описанный эксперимент проводился на ноутбуке MacBook Pro с процессором Apple M3 Pro, включающим 11 ядер: 5 производительных и 6 энергоэффективных. Объём оперативной памяти составлял 18 ГБ. Операционная система — macOS 15.0.

Результаты эксперимента

Сводная статистика по большому полю приведена в таблице 2. Каждый алгоритм участвовал в трёх попарных сравнениях, поэтому суммарно для него учитывалось 150 партий.

Наибольшую долю побед показал метод альфа-бета-отсечений. Показатели вычислительной стоимости приведены в таблице 3.

Таблица 3. Вычислительные показатели на поле 11×11

Показатель	Случайный	Минимакс	Монте-Карло	Альфа-бета-отсечения
Длина партии, ходов	114,87	111,43	120,27	107,17
Время хода, мс	1,0	1578,5	6576,2	2743,9
Глубина поиска	1,00	3,96	108,60	6,71
Состояний просмотрено	4,4	2158,1	444,7	5405,1
Ветвей отброшено	0,0	0,0	0,0	1140,3

Метод Монте-Карло тратит больше всего времени на ход, поскольку выполняет множество симуляций. Метод минимакса работает быстрее, но не использует отсечения. Метод альфа-бета-отсечений требует больше времени, чем метод минимакса, зато просматривает более глубокие продолжения и отбрасывает часть ветвей дерева.

Заключение

В работе рассмотрен модуль программного соперника для игрового приложения «Коридор». Описаны игровая модель, функция оценки позиции и три алгоритма выбора хода. Для сравнения алгоритмов использовались автоматические партии.

Эксперимент показал, что среди реализованных стратегий наибольшую долю побед получил метод альфа-бета-отсечений. Он использует отсечения и лучше подходит для более глубокого поиска, тогда как метод минимакса проще и быстрее. Метод Монте-Карло оказался конкурентоспособным по качеству игры, но требует существенно большего времени на ход. Код приложения находится в соответствующем репозитории: <https://github.com/karpuninaleksandr/quoridor>.

Список литературы

1. *Hobby Games*. Коридор. Правила игры. URL: https://hobbygames.ru/download/rules/Quoridor_Rules.pdf (дата обр. 08.06.2026).
2. *Russell S., Norvig P. Artificial Intelligence: A Modern Approach*. Hoboken : Pearson, 2021. 1168 p.
3. *Coulom R. Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search* // *Computers and Games*. 2006. P. 72–83.